Concurrency and Distributed Systems

Week 5 – Part 1: Task Execution and ExecutorService

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 1 Roadmap

1. Why Task Execution Matters
2. From Threads to Executors
3. ExecutorService Lifecycle
4. Runnable, Callable, and Future
5. Timeouts and Bulk Submission
6. Lab Preparation

Why This Part Matters

Core question

How do we execute many independent tasks without creating an unlimited number of threads?

Week 5 moves from **thread mechanics** to **execution policy**.

- Week 2: create, start, interrupt, and join threads.
- Week 3: protect shared state with monitors and locks.
- Week 4: make writes visible and safely publish state.
- Week 5: run many tasks with bounded resources.

A Task

Definition

A task is a discrete unit of work with a clear beginning, end, input, and side effect or result.

Good task boundaries

- Handle one HTTP request.
- Process one file.
- Compute one quote.
- Update one metric snapshot.

Bad task boundaries

Hidden shared state and unbounded lifetime make execution policy impossible to reason about.

Sequential Execution

```
while (true) {
    Socket connection = server.accept();
    handleRequest(connection);
}
```

- Easy to understand.
- Only one request is handled at a time.
- One slow request delays every later request.
- CPU may sit idle while the handler waits for I/O.

Thread Per Task

```
while (true) {
    Socket connection = server.accept();
    new Thread(() -> handleRequest(connection)).start();
}
```

- Improves responsiveness under light load.
- Work can proceed while other tasks block on I/O.
- Thread creation is now tied directly to input rate.

Problem

If requests are unbounded, threads are unbounded.

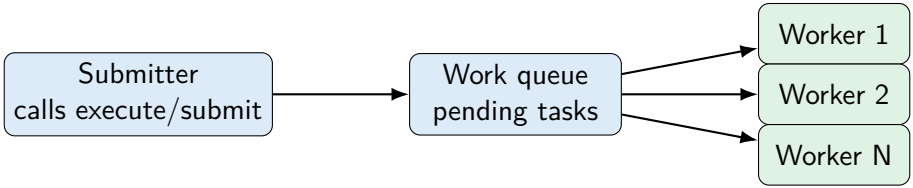
The Thread Explosion Problem

- Each platform thread consumes memory.
- Too many runnable threads increase context switching.
- Too many blocked threads still consume resources.
- Failure mode: the JVM or OS refuses to create more threads.

Engineering rule

Resource usage must be bounded by design, not by accident.

What Executor Changes



Key idea

The caller submits work; the executor chooses when, where, and how that work runs.

Executor

```
1 public interface Executor {
2     void execute(Runnable command);
3 }
4
5 Executor executor = Executors.newFixedThreadPool(4);
6 executor.execute(() -> handleRequest());
```

- Minimal interface.
- Accepts `Runnable`.
- Does not expose results.
- Does not expose shutdown directly.

ExecutorService

```
ExecutorService pool = Executors.newFixedThreadPool(4);

Future<Integer> future =
    pool.submit(() -> computeScore());

pool.shutdown();
```

- Extends `Executor`.
- Adds `submit()`, `invokeAll()`, `invokeAny()`.
- Adds lifecycle control: `shutdown()`, `shutdownNow()`, `awaitTermination()`.
- Returns `Future` handles for result, completion, and cancellation.

Execution Policy

An executor encodes policy

It answers operational questions that direct thread creation does not.

- How many tasks may run at once?
- Where do waiting tasks go?
- In what order are tasks processed?
- What happens when the system is full?
- How does shutdown behave?

Demo04: Fixed Thread Pool

```
ExecutorService pool = Executors.newFixedThreadPool(2);
AtomicInteger completed = new AtomicInteger();

for (int i = 0; i < tasks; i++) {
    pool.submit(completed::incrementAndGet);
}

pool.shutdown();
pool.awaitTermination(2, TimeUnit.SECONDS);
```

What students should notice

Five submitted tasks can run on two reusable worker threads.

Executor Lifecycle

Method	Meaning
`shutdown()`	Stop accepting new tasks; finish queued/running tasks.
`shutdownNow()`	Try to interrupt running tasks and return queued tasks.
`awaitTermination()`	Wait for termination after shutdown request.
`isShutdown()`	Has shutdown been requested?
`isTerminated()`	Are all tasks finished after shutdown?

Common mistake

Forgetting shutdown keeps worker threads alive and can keep a program running.

Runnable

```
Runnable task = () -> {
    updateCounter();
};

Future<?> f = pool.submit(task);
```

- `run()` returns `void`.
- Useful for side effects.
- The `Future<?>` can still represent completion or cancellation.
- Exceptions escape the task and are reported through the `Future`.

Callable

```
1 Callable<Integer> task = () -> {
2     return computeScore();
3 };
4
5 Future<Integer> f = pool.submit(task);
```

- `call()` returns a value.
- May throw checked exceptions.
- `Future<T>` preserves the result type.

Runnable vs Callable

Feature	`Runnable`	`Callable<T>`
Method	`run()`	`call()`
Return value	none	`T`
Checked exception	no	yes
Typical use	side effect	computed result
Submit result	`Future<?>`	`Future<T>`

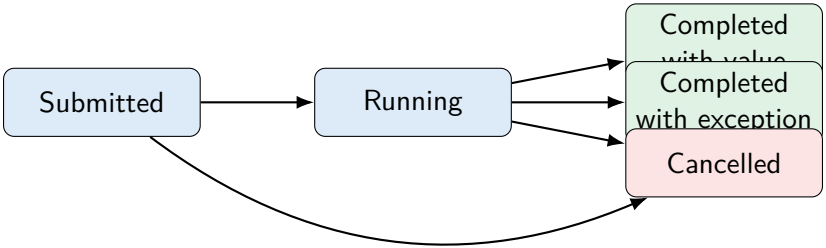
Demo14: Callable and Future

```
ExecutorService pool = Executors.newSingleThreadExecutor();
try {
    Callable<Integer> work = () -> input * input;
    Future<Integer> result = pool.submit(work);
    return result.get(1, TimeUnit.SECONDS);
} finally {
    pool.shutdownNow();
}
```

What students should notice

`submit()` starts asynchronous work; `get()` is where the caller joins the result.

Future States



Future is a handle

It is not the result itself; it is a way to observe or influence task completion.

Result Retrieval: Three Styles

```
T value = future.get(); // waits until done

while (!future.isDone()) {
    // bad: polling burns CPU or adds arbitrary sleep
}

T value = future.get(500, TimeUnit.MILLISECONDS);
```

- Blocking `get()` is simple when waiting is acceptable.
- Polling with `isDone()` is usually an anti-pattern.
- Timeout-aware `get()` is better when the caller has a time budget.

Handling Failure and Interruption

```
try {
    return future.get(timeout, unit);
} catch (TimeoutException e) {
    future.cancel(true);
    throw e;
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    future.cancel(true);
    throw e;
} catch (ExecutionException e) {
    throw unwrap(e);
}
```

Important

Do not swallow interrupts. Reassert the interrupt status when you cannot handle it locally.

Bulk Submission with invokeAll

```
List<Future<Quote>> futures =
    pool.invokeAll(tasks, 1, TimeUnit.SECONDS);

for (Future<Quote> f : futures) {
    if (!f.isCancelled()) {
        Quote q = f.get();
    }
}
```

- Submit a batch of independent tasks.
- Bound the total waiting time.
- Cancel unfinished tasks when the time budget expires.

Where invokeAny Fits

- `invokeAny()` returns the first successful result.
- Useful for redundant providers or fastest-answer-wins logic.
- Remaining tasks are cancelled after a successful result.

Example

Ask several quote providers, use the first valid quote, cancel the rest.

What This Lecture Does Not Solve Yet

- It does not decide queue capacity.
- It does not decide pool size.
- It does not decide overload behavior.
- It does not make shared state automatically safe.

Bridge

Those are the next Week 5 topics: backpressure, pools, atomics, locks, and cleanup.

Lab Preparation

- Read `Demo04_FixedThreadPool`.
- Read `Demo14_CallableAndFuture`.
- In every executor example, identify:
  - worker count,
  - queue behavior,
  - shutdown path,
  - result retrieval path.

Part 1 Takeaway

Final takeaway

ExecutorService is not just a convenience API. It is the place where a program makes task execution policy explicit.

Bridge to Part 2

If submission is easy, overload becomes easy too.

The next question is:

How do we stop producers from flooding the system?

End of Part 1

Questions?